

Design document for API01 metadata and data transfer API (D1.1)

The ETHZ ORD Measure 2 API-01 Project team

Introduction.....	1
Requirements	1
Technical design	2
Data and metadata format (1,2,3,6,7,8,9)	2
Mapping metadata elements across systems.....	4
Approach 1: Agreeing on a common namespace and terms	4
Approach 2: Expressing metadata as RDF and defining equivalences with elements from a shared ontology.....	5
Approach 3: Hybrid of 1 and 2 (Current agreement)	6
Protocol definition (4).....	11
API definition (5).....	12

Introduction

This document outlines the requirements and design choices for developing a data exchange protocol between data catalogues/ELNs. It is designed for the practical use case of transferring data and metadata between [openBIS](#) and [SciCat](#), which will serve as initial test cases for development. The goal is to establish a foundation for future data and metadata exchange with other ELNs and data catalogues.

Requirements

The solution should enable consistent data and metadata transfer, allowing to share data together with its metadata description, ideally:

1. in a single bundle
2. in an agreed and widespread file format
3. following an agreed metadata model
4. In an agreed protocol
5. Expose agreed APIs
6. Agree on a shared [RDF](#) ontology (schema.org/datacite) and share system specific data by expressing their semantical meaning through equivalences to the shared RDF ontology
7. Each metadata field should have a clear semantic definition
8. Should allow sharing one or multiple files/folders together and support large files.
9. In addition, every data sharing system should describe the attached files with a minimum and agreed set of metadata but be free to add any additional metadata, not necessarily agreed upon between all involved systems. In this way, we conceptually decouple the problem of describing a file/dataset and processing it. Processing it semantically will be limited to the common metadata (at a minimum), but this will not limit its description.

This is required to foster standardization, and keeping data and metadata together removes the need to maintain a database with reciprocal links.

Technical design

We divide here in schematic subsections, outlining the design for:

1. Data and metadata format
2. Protocol definition
3. API definition

For each section, we reference in the section heading the involved requirements.

Data and metadata format (1,2,3,6,7,8,9)

Many solutions are available ([RO-Crate](#), [METS](#), [OAI-ORE](#), Datacite, [Dublin Core](#), [OAI-PMH](#), ...). Out of these, RO-Crate seems the strongest candidate as it is growing very fast, has an active community and meets all the requirements. It also embraces [FAIR](#) principles. It is based on RDF and built with [JSON-LD](#) in mind. By using RO-Crate, we inheritably set the basis to solve 1,2,3,7,8 and leave space for metadata enrichment as described in 9.

In essence, [RO-Crate](#) is a glorified zip file (1,2,8) containing files (data containers) and a JSON-LD compliant file (3,7,9), called `ro-crate-metadata.json`, which contains metadata referencing and describing the files inside the zip. Being based on JSON-LD (3), it allows to semantically define every metadata field or entity (7), leaving space for metadata enrichment, by adding custom, semantically annotated definitions in the `ro-crate-metadata.json` file (9). The zip container enables having data and metadata together (1), sets zip as the data format (2), and, together with the RO-Crate definition, enables referencing and describing files and folders either inside the zip or through their remote URL (e.g. their [Object Store](#) or HTTP object URL – API02 –), thus enabling multiple, potentially large, file sharing (8).

Well-known platforms already make use of JSON-LD, some implement it as a format for metadata export, notably [Zenodo](#) includes it as one of its export formats. While this allows sharing high level metadata, we believe RO-Crate could add, when needed and without major changes, the ability to:

- Define and validate a JSON-LD shape, by defining and using a [RO-Crate profile](#). The profile, when agreed upon, can encode the shape of the transferred object, and mark fields as required, mandatory or recommended together with other semantic information.
- With the Profile, we can add guidelines on how to include system specific metadata, not necessarily shared by all adopting partners.

- Eventually encode more complex relations between entities, by including them in the Profile definition.
- Add the ability to include files together with their metadata, by referencing their relative path in the RO-Crate. This will extend the current limitation of schema.org which only supports including remote links to files as “ [DataDownload](#)” elements. This is central to the project, at least in its early phase, since not all involved systems expose data via remote accessible URLs. This also implicitly removes one additional requirement from adopting partners.

In summary, when comparing RO-Crate to raw JSON-LD, the former seems particularly interesting as it allows to include files in the crate directly, not limiting them to schema.org DataDownload and enables schema definition via RO-Crate Profiles.

For data catalogues and ELNs interoperability, we need a way to share system specific data, such that it can be interpreted by another system in a meaningful way. (6). The next section describes three approaches to achieve this. In the first approach, participating systems agree on a minimal metadata schema, e.g. the Datacite metadata schema adopted as per API-03. Whereas in the second approach, the systems export metadata with their own schemas, but express equivalence between their schema and elements from the Datacite ontology using [OWL](#)/RDF. The third approach is a combination of approach 1 and 2, where adopting facilities are free to select 1 and 2 based on the needs and it is a responsibility of the interoperating systems to make sense of the equivalences, if any. All approaches use JSON-LD / RO-Crate as the language for the export and will largely benefit from the Datacite definitions from API-03.

The systems in question evolved separately and each has their own schema. Transferring the schema together with the metadata allows exporting the metadata as it is expressed in the system. Each importing system can use the bundled schema in adapting the data to their own representation. In particular, openBIS is a system with a dynamic schema, thus the schema from the interchange format can be imported and used alongside the metadata inside it.

Mapping metadata elements across systems

While RO-Crate / JSON-LD serves as the container format, it remains to be defined how the participating systems interoperate. In JSON-LD, the [@context](#) entity defines how keys in the JSON document map to URIs. RO-Crate specification uses (slightly modified) schema.org context, hence all keys map to schema.org vocabulary / ontology.

In our system, the schema.org vocabulary is insufficient to unambiguously express all metadata, hence there is need to support custom terms. The first two approaches we

present below differ primarily in how much semantic information is present about these custom terms. In Approach 1, the terms are simply URIs with no further properties, whereas in Approach 2, the terms come from an agreed ontology (i.e. the terms are RDF resources e.g. `rdfs:Classes` or `rdfs:Properties` with domain and range constraints). This enables us to define the schema of our metadata in a standard way, in contrast with Approach 1, where the schema was implicit / agreed upon. Approach 3 is a combination of 1 and 2 and allows leveraging the advantages of the two, i.e. the extensibility provided by 2 and the simplicity from 1, when no additional semantic information is required.

Approach 1: Agreeing on a common namespace and terms

In this approach, participating systems agree on a set of terms and the shapes (i.e. types) of the entities they wish to exchange (i.e. they agree on a minimal metadata schema). This could be achieved by extracting such terms to a common namespace, thus creating URIs for each of these terms. In addition, systems may include their own terms in the export. For example, openBIS may include `openBIS:collection` class and its instances in the export, and SciCat and other catalogues are free to process it in an appropriate way or ignore it.

The importing system defines its behavior only for the commonly agreed terms/entities. For the sake of concreteness, an exported JSON-LD from openBIS might look like the following:

```
{
  ...
  creator: {@id: #creator1},
  publicationYear: 2024,
  ...
  {
    @id: #creator1,
    creatorName: "Jane Doe"
    ...
  },
  openBIS:collectionName: "sample"
}
```

Pros: relatively less effort,
Cons: does not support properties expansion. For example, it does not support adding semantical information, system specific, to the agreed terms.

Approach 2: Expressing metadata as RDF and defining equivalences with elements from a shared ontology

In this approach, systems export data in terms of their own domain specific schema. However, they can define (and optionally include in the export) the schema itself and a mapping to terms from a common *ontology*. This schema is defined in a standard way using RDFS and the mapping is done in a standard way using OWL e.g. `owl:equivalentClass`, `owl:equivalentProperty`.

```
{
  ...
  openBIS:creator: {@id: #creator1},
  openBIS:year: 2024,
  ...
  {
    @id: #creator1,
    @type: openBIS:creator
    openBIS:name: "Jane Doe"
    ...
  },
  openBIS:collectionName: "sample",
  {
    @id: openBIS:creator,
    @type: rdfs:class,
    owl:equivalentClass: schema.org:creator
  }
}
```

This export differs from the export in Approach 1 in two main ways:

1. The custom schema / type (i.e. the openBIS schema) is itself expressed in RDFS. Here, the schema / type definition is included in the export. However, the schema definition can also live in a separate `.jsonld` or `.rdf` file and included in the export in the actual implementation.
2. The `schema.org` terms now point to an ontology, hence `"https://schema.org/creator"` is a `rdfs:Class`, instead of just a plain URI.

While this approach has disadvantages of being more complex (e.g. an OWL reasoner will be needed to extract the equivalences), it offers the following advantages compared to Approach 1:

- It decouples semantic definition from data representation (i.e. semantic definition is not contained in the data but rather in the types definition)
- It allows extending existing ontologies directly in the types definition.

Approach 3: Hybrid of 1 and 2 (Current agreement)

In this approach, systems can export data using:

- Common RO-Crate vocabulary (defined in the [RO-Crate Context](#))
- Custom vocabulary
- A mix of both

This flexibility permits re-using already defined vocabulary where it makes sense as well as defining custom terms for more system-specific metadata.

We recommend using as much as possible RO-Crate vocabulary and only defining equivalent custom terms if it adds semantic information.

If the same information is expressed in both common AND custom vocabulary, importing systems might duplicate the information.

The produced RO-Crate should be self-contained and not rely on external URLs like the RO-Crate context, therefore every term re-used should be explicitly added to the prefix map. This is done to prevent problems if the referenced URLs are discontinued. We allow reference to schema.org, thus not explicitly including its terms, given its stability and widespread use.

For instance, to use the “name” property from schema.org:

```
{
  "@context": [
    "https://w3id.org/ro/crate/1.1/context",
    {
      "name": "https://schema.org/name"
    }
  ],
  "@graph": [
    {
      ...
      "name": "Example name"
      ...
    }
  ]
}
```

Custom vocabulary should be defined according to this [specification](#) and included in the top-level “hasPart” property.

To summarize:

- Class definitions should be of type “rdfs:Class”
- Classes may define equivalent classes using “owl:equivalentClass” property

- Property definitions should be of type “rdfs:Property”
- Properties may define equivalent classes using “owl:equivalentProperty” property
- For a custom property to be intelligible by other systems it should be equivalent to a standard RO-Crate term (Schema.org). For all terms where the schema.org definition is not sufficient, we suggest prioritizing the datacite ontology to express the equivalence. Being equivalent to more than one term is allowed.
- Property must describe in which types they can be used with the “schema:domainIncludes” property
- Property must describe the types they can hold using the “schema:rangeIncludes” property

Example of a system specific term definition:

```
{
  "@type": "rdfs:Property",
  "@id": "PUBLICATION_YEAR",
  "schema:rangeIncludes": { "@id": "xsd:dateTime" },
  "schema:domainIncludes": { "@id": "PUBLICATION" },
  "owl:equivalentProperty": [
    { "@id": https://schema.datacite.org/meta/kernel-4/metadata.xsd#publicationYear },
    { "@id": https://schema.org/datePublished }
  ]
}
```

Here the “PUBLICATION_YEAR” term is defined as an RDFS Property of type “xsdDatetime”. It is expected to be present on individuals of type “PUBLICATION” and is equivalent to both [Schema.org datePublished](https://schema.org/datePublished) and [DataCite's publicationYear](https://schema.datacite.org/meta/kernel-4/metadata.xsd#publicationYear).

We foresee two main use cases for using system specific term definitions. Whenever the publishing system needs to add additional semantic information, in the example the

“schema:rangeIncludes” info, to an existing equivalent term with owl:equivalentProperty, and when it requires defining specific properties where existing ontologies do not fit.

Even with the flexibility provided by approach 3, adopting partners need to agree on a common vocabulary for the shared terms. We start with Publications, i.e. the container of data published as part of an experiment. Other types could follow in the course of the project. As mentioned before, we believe an RO-Crate Profile could be of great help here.

Exchanging Publication metadata:

- To easily find publications in the graph, the class should be equivalent or a sub class of [CreativeWork](#)
- Required and optional properties for a Publication are as defined in API-03, as well as the mapping between Datacite and schema.org.

For [publications](#):

Schema.org property		Default
identifier	Mandatory	
creator	Mandatory	
name	Mandatory	
publisher	Strongly recommended	Importing system
datePublished	Strongly recommended	Current date
license	Strongly recommended	https://creativecommons.org/licenses/by-sa/4.0/
description	Strongly recommended	Title
inLanguage	Strongly recommended	eng
hasPart	Mandatory	
keywords	Recommended	
size	Recommended	

For [Creator](#):

Schema.org property		
name	Mandatory	
familyName	Strongly recommended	Name[1]
givenName	Strongly recommended	Name[0]
affiliation	Strongly recommended	Importing system

Approach 3 is the currently agreed candidate for the implementation.

Protocol definition (4)

Information transfer needs to go through a public network as in most cases ELNs and data catalogues are hosted inside different local networks. For simplicity, we agree on HTTPS.

API definition (5)

We set the basis here for describing the protocol interface, i.e. its API. This is needed to abstract the application logic from its public interface and allow consumers to interact with it. An example is the definition of how to upload a RO-Crate, which could happen using POST /RO-Crate, with the payload being the RO-Crate zip file. Already here, we define the /path, the HTTP verb and the payload. During the application implementation, we should provide an openAPI.json descriptor. The final definition will be modified during the project.

At a minimum, the application should:

- Be able to connect to a data catalogue/ELN
- Enable the download of a RO-Crate, compliant with the Data and metadata section, by specifying the entity of the data catalogue/ELN.
 - For example, when connected to SciCat, the user could call /ro-crate/download/123. This will download the RO-Crate generated from the SciCat dataset 123, with corresponding metadata. The Files will either be in the ZIP file, if under a certain file size limit, or preliminary uploaded to HTTP and referenced in the RO-Crate (if not already available)
 - Similarly, when connected to openBIS, it should do the same but for an openBIS entity.
- Enable the upload of RO-Crates.
 - When connected to SciCat, this will create the specified metadata entity in SciCat. The service will understand what entities to create depending on the ro-crate-metadata.json description (e.g. a RO-Crate Dataset to a SciCat Dataset).
 - Similarly, when connected to openBIS, it should do the same but from an openBIS entity.

The complete openAPI definition can be found [here](#) and represents deliverable D1.2.